

Data Structures and Algorithm Analysis

Quiz 3 Practice

10-607

1. Given a binary search tree with integer valued nodes, provide an algorithm to recover a sorted list of elements. In your description of your algorithm, prove both correctness and time-complexity with the invariants of the BST data structure.
2. Provide the recurrences and runtime complexity (big-O) of the following algorithms:
 - (a) Algorithm A solves a problem of size n by dividing it into 4 subproblems, each of size $n/3$, solving each subproblem, and then combining the solutions in linear time.
 - (b) Algorithm B solves a problem of size n by dividing it into nine subproblems, each of size $n/2$, solving each subproblem, and then combining the solutions in quadratic time.
3. We are given a linked list of distinct integers.
 - a) Suppose that the linked list is sorted. Provide an $O(1)$ time algorithm to construct a binary search tree from this linked list. What is the lookup time of any element in the resultant search tree?
 - b) Suppose that the list is not sorted. What is the time complexity of constructing a binary search tree from this list?
 - c) We say a tree is *balanced* when the left and right subtree height/depth of each node differs by at most 1. Discuss how we can implement a binary tree interface that has this invariant built-in.
4. Provide an algorithm to reverse a queue in $O(N)$ time complexity and $O(1)$ space complexity. Suppose we are on the client side, so you may not assume anything about the underlying implementation of the queue (e.g arrays or linked lists or doubly linked lists). You only have access to the following functions: $\text{deq}(q)$, $\text{enq}(q, \text{elem})$, and $\text{size}(q)$.

Follow up: Now, assume we are on the library side of the implementation. We want to create a function called $\text{reverse}(q)$ that reverses a given queue in $O(N)$ time complexity and $O(1)$ space complexity. We know that our queue is implemented with a singly-linked list. Provide your algorithm.
5. Suppose we have a undirected graph $G = (V, E)$. Let $s, t \in V$ be two arbitrary vertices. We wish to discover whether an $s-t$ path exists in G . Provide an algorithm that checks for an $s-t$ path in G . Your algorithm should run in $O(V + E)$ time.

Follow-up question: now suppose we want to see if there exists a $s - t$ path that uses less than k edges. How can we see if there is such a path?

6. Consider the following undirected weighted graph $G = (V, E)$ with vertices

$$V = \{A, B, C, D, E, F\}$$

and edges (with weights in parentheses):

$$E = \{AB(4), AC(2), BC(1), BD(5), CD(8), \\ CE(10), DE(2), DF(6), EF(3)\}.$$

- (a) Use **Kruskal's algorithm** to compute a minimum spanning tree (MST) of G . Show the order in which edges are considered and which edges are added or skipped. State the final MST and its total weight.
- (b) Use **Dijkstra's algorithm** to compute the shortest-path distances from source vertex A to all other vertices. Show the distances after each iteration when a new vertex is finalized. Draw (or list) the final shortest-path tree rooted at A .